

11 Illustrate the concept of multithreading using a C program.

```
#include <stdio.h>
#include <pthread.h>

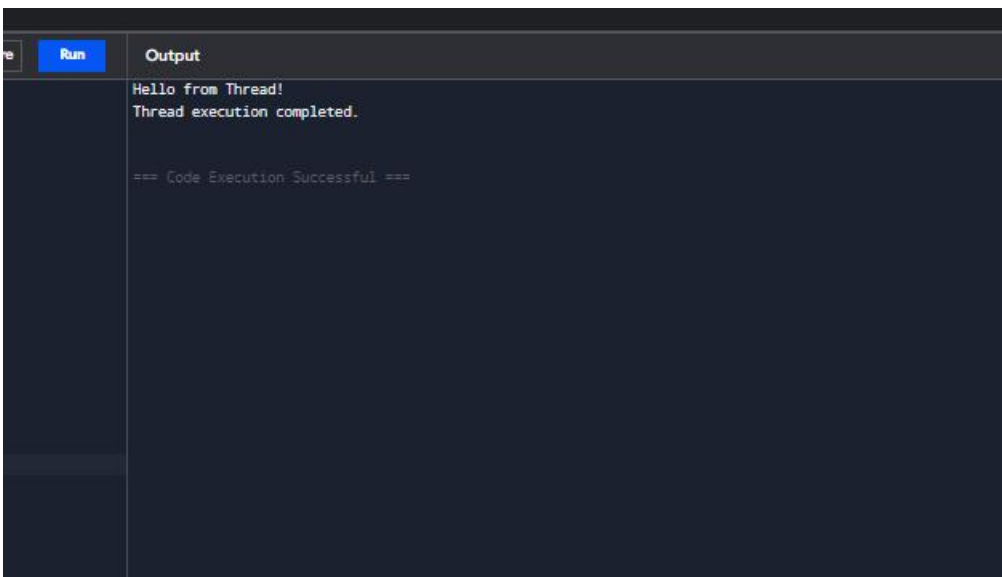
void *display(void *arg) {
    printf("Hello from Thread!\n");
    return NULL;
}

int main() {
    pthread_t t;

    pthread_create(&t, NULL, display, NULL); // Create thread
    pthread_join(t, NULL);                  // Wait for thread

    printf("Thread execution completed.\n");

    return 0;
}
```



The screenshot shows a code editor interface with a dark theme. On the left, there is a sidebar with a 'Run' button. The main area is divided into two panes. The top pane is titled 'Output' and contains the following text: 'Hello from Thread!', 'Thread execution completed.', and '=== Code Execution Successful ==='. The bottom pane is empty.

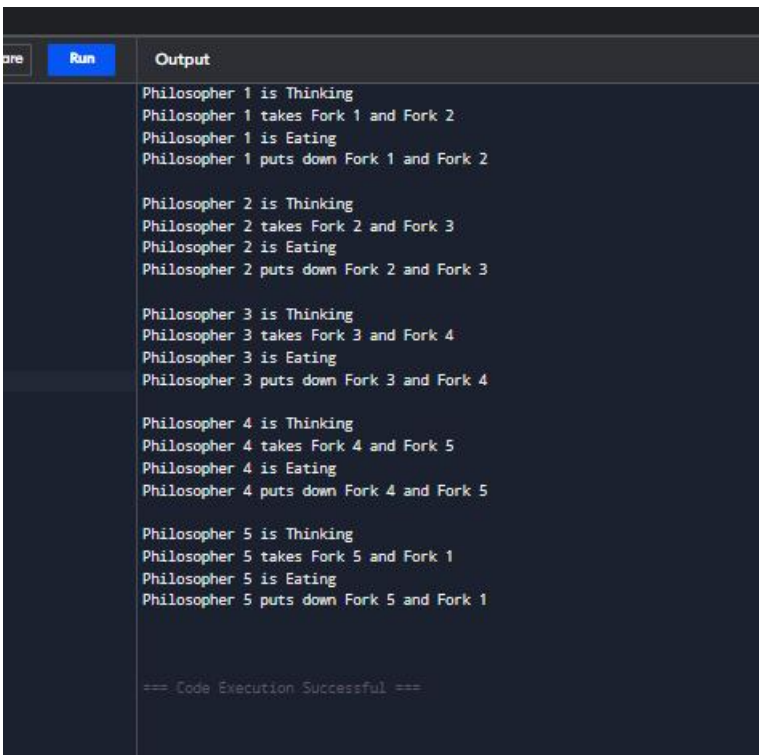
12 Design a C program to simulate the concept of Dining-Philosophers problem

```
#include <stdio.h>

int main() {
    int i;

    for(i = 1; i <= 5; i++) {
        printf("Philosopher %d is Thinking\n", i);
        printf("Philosopher %d takes Fork %d and Fork %d\n", i, i, (i % 5) + 1);
        printf("Philosopher %d is Eating\n", i);
        printf("Philosopher %d puts down Fork %d and Fork %d\n\n", i, i, (i % 5) + 1);
    }

    return 0;
}
```

A screenshot of a code editor showing the output of a C program. The output is displayed in a dark-themed window with a 'Run' button. The text shows the sequence of actions for five philosophers: thinking, taking two forks, eating, and putting down the forks. The sequence repeats for each philosopher from 1 to 5. At the bottom, it says '=== Code Execution Successful ==='.

```
Philosopher 1 is Thinking
Philosopher 1 takes Fork 1 and Fork 2
Philosopher 1 is Eating
Philosopher 1 puts down Fork 1 and Fork 2

Philosopher 2 is Thinking
Philosopher 2 takes Fork 2 and Fork 3
Philosopher 2 is Eating
Philosopher 2 puts down Fork 2 and Fork 3

Philosopher 3 is Thinking
Philosopher 3 takes Fork 3 and Fork 4
Philosopher 3 is Eating
Philosopher 3 puts down Fork 3 and Fork 4

Philosopher 4 is Thinking
Philosopher 4 takes Fork 4 and Fork 5
Philosopher 4 is Eating
Philosopher 4 puts down Fork 4 and Fork 5

Philosopher 5 is Thinking
Philosopher 5 takes Fork 5 and Fork 1
Philosopher 5 is Eating
Philosopher 5 puts down Fork 5 and Fork 1

=== Code Execution Successful ===
```

17. Illustrate the deadlock avoidance concept by simulating Banker's algorithm with

C.

```
#include <stdio.h>
```

```
int main() {
    int n, m, i, j, k;
```

```
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter number of resource types: ");
scanf("%d", &m);
int alloc[n][m], max[n][m], need[n][m];
int avail[m];
int finish[n], safeSeq[n];
printf("\nEnter Allocation Matrix:\n");
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        scanf("%d", &alloc[i][j]);
    }
}
printf("\nEnter Maximum Matrix:\n");
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        scanf("%d", &max[i][j]);
    }
}
printf("\nEnter Available Resources:\n");
for(i = 0; i < m; i++) {
    scanf("%d", &avail[i]);
}
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}
```

```

}
for(i = 0; i < n; i++)
    finish[i] = 0;
int count = 0;
while(count < n) {
    int found = 0;
    for(i = 0; i < n; i++) {
        if(finish[i] == 0) {
            int possible = 1;
            for(j = 0; j < m; j++) {
                if(need[i][j] > avail[j]) {
                    possible = 0;
                    break;
                }
            }
            if(possible) {
                for(k = 0; k < m; k++)
                    avail[k] += alloc[i][k];
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
}
if(found == 0) {
    printf("\nSystem is NOT in a Safe State.\n");
    return 0;
}

```

```

    }
}

printf("\nSystem is in Safe State.\n");
printf("Safe Sequence: ");
for(i = 0; i < n; i++) {
    printf("P%d", safeSeq[i]);
    if(i != n - 1)
        printf(" -> ");
}
printf("\n");
return 0;
}

```

The screenshot shows a code editor with a 'Run' button and an 'Output' window. The output window displays the following text:

```

Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is in Safe State.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2

=== Code Execution Successful ===

```

18. Construct a C program to simulate producer-consumer problem using semaphores.

```
#include <stdio.h>
```

```
int mutex = 1;
```

```
int full = 0;
```

```
int empty = 3, x = 0;
```

```
int wait(int s)
```

```
{
```

```
    return (--s);
```

```
}
```

```
int signal(int s)
```

```
{
```

```
    return (++s);
```

```
}
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    while (1)
```

```
    {
```

```
        printf("\n1. Producer");
```

```
        printf("\n2. Consumer");
```

```
        printf("\n3. Exit");
```

```
        printf("\nEnter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice){
```

```
            case 1:
```

```
                if ((mutex == 1) && (empty != 0))
```

```

{
    mutex = wait(mutex);
    full = signal(full);
    empty = wait(empty);
    x++;
    printf("Producer produces item %d\n", x);
    mutex = signal(mutex);
}
else
{
    printf("Buffer is Full!\n");
}
break;
case 2:
if ((mutex == 1) && (full != 0))
{
    mutex = wait(mutex);
    full = wait(full);
    empty = signal(empty);
    printf("Consumer consumes item %d\n", x);
    x--;
    mutex = signal(mutex);
}
else
{
    printf("Buffer is Empty!\n");
}

```

```
        break;

    case 3:

        return 0;

    default:

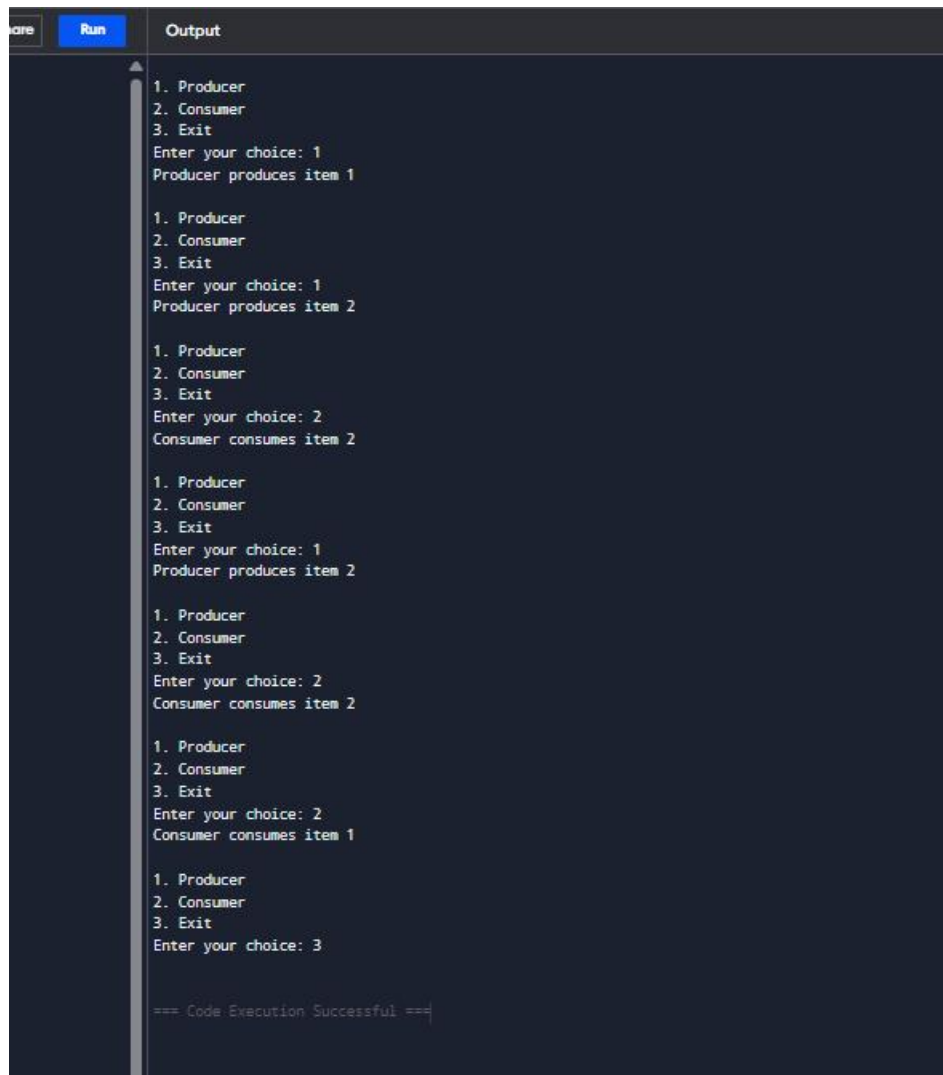
        printf("Invalid Choice!\n");

    }

}

return 0;

}
```



```
Run Output
1. Producer
2. Consumer
3. Exit
Enter your choice: 1
Producer produces item 1

1. Producer
2. Consumer
3. Exit
Enter your choice: 1
Producer produces item 2

1. Producer
2. Consumer
3. Exit
Enter your choice: 2
Consumer consumes item 2

1. Producer
2. Consumer
3. Exit
Enter your choice: 1
Producer produces item 2

1. Producer
2. Consumer
3. Exit
Enter your choice: 2
Consumer consumes item 2

1. Producer
2. Consumer
3. Exit
Enter your choice: 2
Consumer consumes item 1

1. Producer
2. Consumer
3. Exit
Enter your choice: 3

=== Code Execution Successful ===
```

19. Design a C program to implement process synchronization using mutex locks.

```
#include <stdio.h>

#include <pthread.h>

int counter = 0;

pthread_mutex_t lock;

void *increment(void *arg)
{
    int i;

    pthread_mutex_lock(&lock);

    for(i = 1; i <= 5; i++)
    {
        counter++;

        printf("Counter = %d\n", counter);
    }

    pthread_mutex_unlock(&lock);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

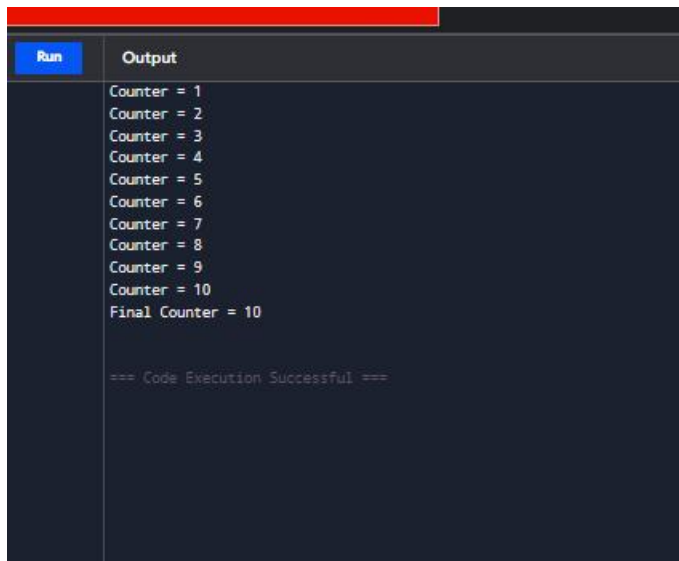
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock);

    printf("Final Counter = %d\n", counter);
}
```

```
    return 0;
}
```



```
Run    Output
Counter = 1
Counter = 2
Counter = 3
Counter = 4
Counter = 5
Counter = 6
Counter = 7
Counter = 8
Counter = 9
Counter = 10
Final Counter = 10

=== Code Execution Successful ===
```

20. Construct a C program to simulate Reader-Writer problem using Semaphores.

```
#include <stdio.h>
```

```
int mutex = 1, wrt = 1;
```

```
int readcount = 0;
```

```
int wait(int s)
```

```
{
```

```
    return --s;
```

```
}
```

```
int signal(int s)
```

```
{
```

```
    return ++s;
```

```
}
```

```
void reader(int id)
```

```
{
```

```
    mutex = wait(mutex);
```

```

    readcount++;
    if (readcount == 1)
        wrt = wait(wrt);
    mutex = signal(mutex);
    printf("Reader %d is reading\n", id);
    mutex = wait(mutex);
    readcount--;
    if (readcount == 0)
        wrt = signal(wrt);
    mutex = signal(mutex);
}

void writer(int id)
{
    wrt = wait(wrt);
    printf("Writer %d is writing\n", id);
    wrt = signal(wrt);
}

int main()
{
    int choice, id;
    while (1)
    {
        printf("\n1. Reader");
        printf("\n2. Writer");
        printf("\n3. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
    }
}

```

```
switch (choice)
{
    case 1:
        printf("Enter Reader ID: ");
        scanf("%d", &id);
        reader(id);
        break;
    case 2:
        printf("Enter Writer ID: ");
        scanf("%d", &id);
        writer(id);
        break;
    case 3:
        return 0;
    default:
        printf("Invalid Choice!\n");
}
}
return 0;
}
```

Now ₹399/mo for Students

First year only. Annual plan. billed monthly.
New subscribers only.

Run

Output

```
1. Reader
2. Writer
3. Exit
Enter your choice: 1
Enter Reader ID: 1
Reader 1 is reading
```

```
1. Reader
2. Writer
3. Exit
Enter your choice: 1
Enter Reader ID: 2
Reader 2 is reading
```

```
1. Reader
2. Writer
3. Exit
Enter your choice: 2
Enter Writer ID: 1
Writer 1 is writing
```

```
1. Reader
2. Writer
3. Exit
Enter your choice: 3
```

```
=== Code Execution Successful ===
```